Practica III
(Registros, Contadores y Memorias)

Alex Rafael Soriano Severino
Mecatronica
ITLA
La Caleta, Santo Domingo
20240684@itla.edu.do

En resumen— En esta práctica se desarrollaron múltiples ejercicios orientados al diseño de sistemas combinacionales y secuenciales, aplicando de forma integral conceptos fundamentales de la electrónica digital. Entre las actividades realizadas se encuentran la implementación de funciones lógicas mediante multiplexores, el análisis y diseño de sistemas combinacionales utilizando productos de suma y sumas de productos, la elaboración de tablas de verdad, simplificación algebraica mediante Mapas de Karnaugh, el diseño de máquinas de estado finito utilizando Flip-Flops RS, la deducción de funciones desde diagramas lógicos y temporales, y la implementación de contadores Johnson, anulares y bidireccionales. Además, se estudiaron los registros universales, así como el funcionamiento de los conversores A/D y D/A junto con ejemplos de circuitos comerciales. Todas las implementaciones fueron diseñadas y simuladas utilizando software profesional de electrónica y verificadas en lenguaje Verilog HDL para FPGA. La práctica permitió consolidar conocimientos clave en el diseño digital, reforzando la relación entre teoría, simulación y aplicación real en hardware.

Abstract— This practice involved the design and analysis of a wide range of digital systems, including both combinational and sequential circuits. The activities covered the implementation of logic functions using multiplexers, Boolean simplification through Karnaugh Maps, Finite State Machine (FSM) design with RS flip-flops, truth table extraction from logic diagrams and timing diagrams, and the construction of Johnson and ring counters. Additionally, the operation of universal registers and analog-digital converters (ADC) and digital-analog converters (DAC) was studied, including commercial IC examples. Simulations were conducted using electronic design software, and hardware validation was performed in Verilog HDL on an FPGA development environment. The results confirmed the theoretical analysis and demonstrated the importance of simulation and hardware description languages in modern digital system design.

Palabras claves— Electrónica digital, MUX, Producto de suma, Suma de productos, Flip-Flop, FSM, Verilog, HDL, Johnson Counter, Registro Universal, ADC, DAC, FPGA, Simulación digital.

I. INTRODUCCION

La electrónica digital se basa en la manipulación y el procesamiento de señales binarias a través de dispositivos combinacionales y secuenciales. En el ámbito académico y profesional, el dominio de estos sistemas es fundamental para el desarrollo de soluciones modernas basadas en lógica digital, FPGA y sistemas embebidos.

La presente práctica aborda una serie de ejercicios diseñados para integrar conocimientos clave sobre funciones lógicas, sistemas combinacionales, máquinas de estado, utilización de multiplexores, análisis de diagramas lógicos y temporales, diseño con flip-flops, y generación de contadores específicos. A diferencia de prácticas centradas en un único proyecto, esta práctica presenta un conjunto amplio y variado de ejercicios que fortalecen diferentes áreas del diseño digital.

El proceso incluyó el uso de Mapas de Karnaugh para simplificar funciones, la implementación de circuitos en software de simulación, la descripción en Verilog HDL y la validación en entornos de desarrollo FPGA. De esta forma, se trabajó tanto la parte teórica como la aplicación práctica, permitiendo obtener una comprensión integral del funcionamiento de los sistemas digitales.

II. MARCO TEÓRICO

A. Lógica combinacional

Los circuitos combinacionales generan salidas que dependen exclusivamente de las entradas actuales. No existe memoria ni retención de estados. Entre los dispositivos combinacionales más utilizados se encuentran las compuertas lógicas, multiplexores, demultiplexores, codificadores y decodificadores. La representación habitual se realiza mediante tablas de verdad y expresiones booleanas en forma de suma de productos (SoP) o producto de sumas (PoS).

B. Lógica secuencial

A diferencia de la lógica combinacional, los circuitos secuenciales dependen tanto de las entradas como del estado anterior, utilizando elementos de memoria como los flip-flops. Estos sistemas son utilizados en contadores, FSM, registros y temporizadores.

C. Multiplexores (MUX)

El Multiplexor (MUX) es un circuito combinacional que permite seleccionar una de varias entradas de datos para dirigirla hacia una única salida. Su operación depende de líneas de selección que determinan qué entrada se activa.
Los MUX pueden implementarse en diferentes configuraciones, como 2:1, 4:1 o 8:1, y pueden utilizarse para realizar funciones lógicas de varias variables sin necesidad de compuertas adicionales. Esto los convierte en componentes versátiles en el diseño digital.

D. Flip-Flops

Los flip-flops son celdas de memoria básicas:

- **RS:** controla la salida mediante señales de Set y Reset.
- **D:** copia el valor de la entrada en cada flanco de reloj.
- Otros tipos como JK y T son derivados, pero en esta práctica se usan RS y D.

E. Máquinas de estado finito (FSM)

Una FSM es un modelo secuencial que transita entre estados según sus entradas. Se representa mediante:

- Tabla de estados
- Diagrama de transición
- Ecuaciones de excitación del flip-flop
- Lógica combinacional asociada

F. Registros

Un registro universal es un conjunto de flip-flops que puede operar en múltiples modos: carga paralela, desplazamiento a la izquierda, desplazamiento a la derecha o retención. Se utiliza ampliamente para manipulación de datos, conversión serie-paralelo y procesamiento digital.

G. ADC y DAC

- Un ADC convierte una señal analógica en digital.
- Un DAC realiza el proceso inverso.

Aplicaciones típicas incluyen instrumentación, audio digital, sensores, microcontroladores, procesamiento de señales y comunicación digital.

H. Contadores Johnson y anulares

- El contador anular desplaza un único "1" a través de los flip-flops en bucle.
- El contador Johnson genera una secuencia más larga realimentando la salida invertida.

Estas topologías se emplean en generación de secuencias y distribución de reloj.

III. DESARROLLO DE LOS EJERCICIOS

Ejercicio 1: Se requiere implementar la siguiente función usando un mux 2x1., donde Fw (A,B,C,D) = (1,2,4,5,6,11,12,15). Tome en consideración que se usará producto de suma de dicha función. b. Se requiere conectar un mux 4:1 donde la función es F(A,B,C, D, E) = (0,1,3, 5, 8, 9, 11,14,15, 17, 20, 21, 22,23) + d(25, 27, 30)

Se construyó la tabla de verdad, se identificaron los maxterms correspondientes y se generó la expresión PoS necesaria. Con esta información, la función fue implementada mediante un multiplexor 2:1 seleccionando la variable adecuada como línea de selección. La simulación validó que la salida coincidiera con los valores especificados para los minterms (1,2,4,5,6,11,12,15).

Tabla de VEM:

	S0	VEM			
	A	B	C	D	Fw
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	1
3	0	0	1	1	0
4	0	1	0	0	1
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	0
8	1	0	0	0	0
9	1	0	0	1	0
10	1	0	1	0	0
11	1	0	1	1	1
12	1	1	0	0	1
13	1	1	0	1	0
14	1	1	1	0	0
15	1	1	1	1	1

K-Maps y Funciones:

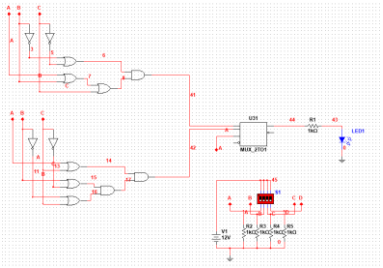
Salida (Fw)				
B/CD	00	01	11	10
0		1		1
1	1	1		1

PoS
(B' + C')(A + B + C)

Salida (Fw)				
B/CD	00	01	11	10
0	0	0	1	0
1	1	0	1	0

PoS
(A + C)(B + C')(B' + C)

Diagrama:



Parte B: Se elaboró la tabla de verdad completa para cinco variables (A,B,C,D,E), incluyendo los minterms donde F = 1 y los términos don't care (25,27,30). La simplificación en PoS permitió obtener una expresión más compacta, aprovechando los don't care para agrupar más términos en el Mapa de Karnaugh. La implementación en software mostró coincidencia total con la tabla generada.

Tabla VEM:

	S0	S1	VEM			
	A	B	C	D	E	F
0	0	0	0	0	0	1
1	0	0	0	0	1	1
2	0	0	0	1	0	0
3	0	0	0	1	1	1
4	0	0	1	0	0	0
5	0	0	1	0	1	1
6	0	0	1	1	0	0
7	0	0	1	1	1	0
8	0	1	0	0	0	1
9	0	1	0	0	1	1
10	0	1	0	1	0	0
11	0	1	0	1	1	1
12	0	1	1	0	0	0
13	0	1	1	0	1	0
14	0	1	1	1	0	1
15	0	1	1	1	1	1

16	1	0	0	0	0	0
17	1	0	0	0	1	1
18	1	0	0	1	0	0
19	1	0	0	1	1	0
20	1	0	1	0	0	1
21	1	0	1	0	1	1
22	1	0	1	1	0	1
23	1	0	1	1	1	1
24	1	1	0	0	0	0
25	1	1	0	0	1	x
26	1	1	0	1	0	0
27	1	1	0	1	1	x
28	1	1	1	0	0	0
29	1	1	1	0	1	0
30	1	1	1	1	0	x
31	1	1	1	1	1	0

K-Maps y Funciones:

Salida (Fw)				
B/CD	00	01	11	10
0	1	1	1	
1		1		

SoP
A'B' + A'C + B'C

Salida (Fw)				
B/CD	00	01	11	10
0	1	1	1	
1			1	1

SoP
A'B' + BC + AB

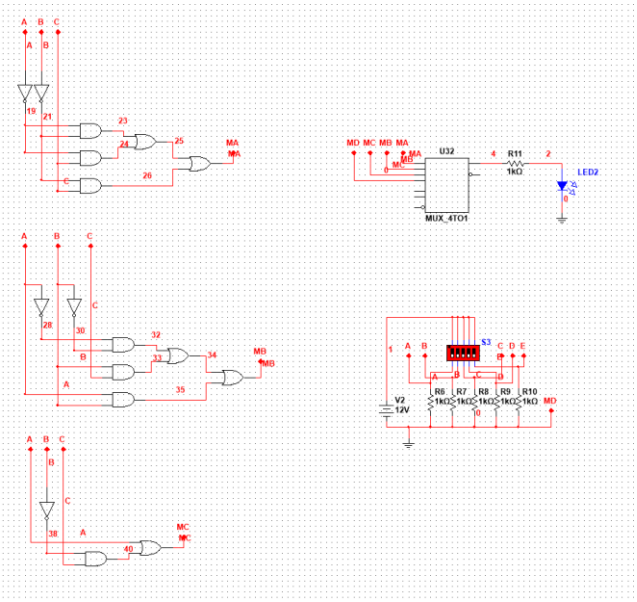
Salida (Fw)				
B/CD	00	01	11	10
0		1		
1	1	1	1	1

SoP
A + B'C

Salida (Fw)				
B/CD	00	01	11	10
0		x	x	
1				x

SoP
0

Diagrama:



Ejercicio 3: Una nave industrial consta de dos portones de entrada (P1,P2) y dos grandes ventanales (V1,V2). Un interruptor accionado por llave (I), indica si se está en jornada laboral o fuera de ella (nave cerrada). Durante la jornada laboral se permite que esté abierto uno de los dos portones pero no los dos al mismo tiempo, pudiendo estar cualquier ventana abierta. Fuera de la jornada laboral, no podrán estar, ni los portones ni las ventanas, abiertas. Diseña un sistema de alarma que detecte estas situaciones. Consideramos que puerta o ventanas abiertas es 0 e interruptor conectado es 1. Resuelva por producto de sumas.

Se describió un sistema capaz de detectar condiciones no permitidas según el estado del interruptor I (jornada laboral).

- En jornada laboral (I=1): está prohibido que ambos portones estén abiertos simultáneamente.
- Fuera de jornada (I=0): ninguna puerta o ventana puede estar abierta.

Se construyó una tabla de verdad con 32 combinaciones y se obtuvo la función en PoS. Finalmente se implementó una versión en HDL que genera la alarma en tiempo real.

Tabla de Verdad:

	I	P1	P2	V1	V2	A
0	0	0	0	0	0	1
1	0	0	0	0	1	1
2	0	0	0	1	0	1
3	0	0	0	1	1	1
4	0	0	1	0	0	1
5	0	0	1	0	1	1
6	0	0	1	1	0	1
7	0	0	1	1	1	1
8	0	1	0	0	0	1

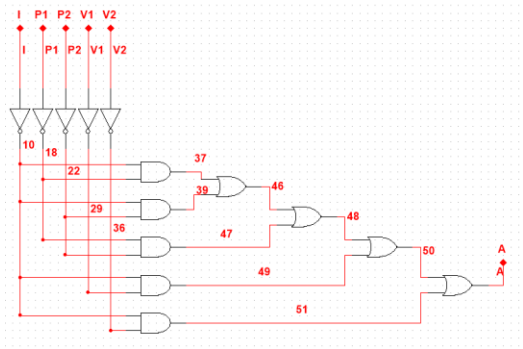
9	0	1	0	0	1	1
10	0	1	0	1	0	1
11	0	1	0	1	1	1
12	0	1	1	0	0	1
13	0	1	1	0	1	1
14	0	1	1	1	0	1
15	0	1	1	1	1	0
16	1	0	0	0	0	1
17	1	0	0	0	1	1
18	1	0	0	1	0	1
19	1	0	0	1	1	1
20	1	0	1	0	0	0
21	1	0	1	0	1	0
22	1	0	1	1	0	0
23	1	0	1	1	1	0
24	1	1	0	0	0	0
25	1	1	0	0	1	0
26	1	1	0	1	0	0
27	1	1	0	1	1	0
28	1	1	1	0	0	0
29	1	1	1	0	1	0
30	1	1	1	1	0	0
31	1	1	1	1	1	0

K-Maps y Funciones:

Salida (A) - A'				
BC/DE	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1		1
10	1	1	1	1

Salida (A) - A				
BC/DE	00	01	11	10
00	1	1	1	1
01				
11				
10				

Diagrama:



Codigo en Verilog:

```
module ejercicio_2(
    input A, B, C, D, E,
    output Sa
);

    assign Sa = ( (~A) | (~C) ) & ( (~A) | (~B) ) & ( (~B) | (~C) | (~D) | (~E) );

endmodule
```

Ejercicio 4: El ingeniero de turno ha estado trabajando en un diseño de un FSM (Finite State Machine) y no ha tenido tiempo para completarlo, así que sin dar muchas explicaciones ha dejado un diagrama de estado y nos ha pedido que realicemos un circuito secuencial con las siguientes características. usando flip flops RS. Además de esto ha dejado una nota que dice use productos de suma para el diseño. Las variables serán Io, I1 e I2 donde Io es la menos significativa.

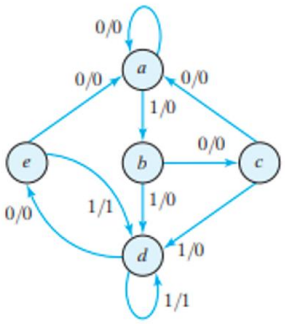


Tabla de estados:

Presente			X	Futuro			Y	R2	S2	R1	S1	R0	S0
A2	A1	A0		A2+	A1+	A0+							
0	0	0	0	0	0	0	0	X	0	X	0	X	0
0	0	0	1	0	0	1	0	X	0	X	0	0	1
0	0	1	0	0	1	0	0	X	0	0	1	1	0
0	0	1	1	0	1	1	0	X	0	0	1	0	X
0	1	0	0	0	0	0	0	X	0	1	0	X	0
0	1	0	1	0	1	1	0	X	0	0	X	0	1
0	1	1	0	1	0	0	0	0	1	1	0	1	0
0	1	1	1	0	1	1	1	X	0	0	X	0	X
1	0	0	0	0	0	0	0	1	0	X	0	X	0
1	0	0	1	0	1	1	1	1	0	0	1	0	1

K-Maps y Funciones:

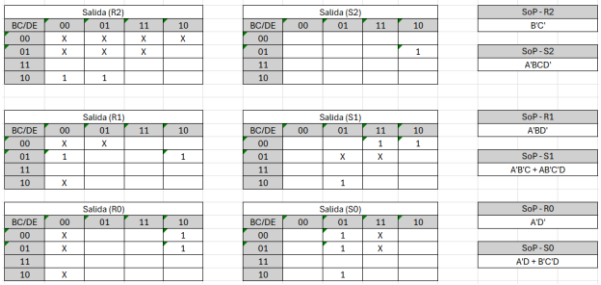
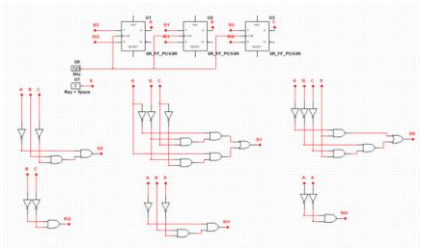


Diagrama:



Codigo en Verilog:

```
module Ejercicio3(
    input A, B, C, X, clk,
    output Q2, Q1, Q0
);

wire S2, R2, S1, R1, S0, R0;

assign S2 = (~A & B & C & ~X);
assign R2 = (~B & ~C);

assign S1 = (~A & ~B & C) | (A & ~B & ~C & X);
assign R1 = (~A & B & ~X);

assign S0 = (~A & X) | (~B & ~C & X);
assign R0 = (~A & ~X);

RS_ff FF2 (S2, R2, clk, Q2);
RS_ff FF1 (S1, R1, clk, Q1);
RS_ff FF0 (S0, R0, clk, Q0);

endmodule
```

Ejercicio 5: De los siguientes diagramas lógicos deduzca la tabla de verdad y escriba la función booleana resultante.

Se analizó el diagrama proporcionado, identificando cada compuerta y su interacción. Se generó la tabla de verdad correspondiente y se obtuvo la ecuación booleana resultante. Esta ecuación fue traducida a un módulo HDL funcional.

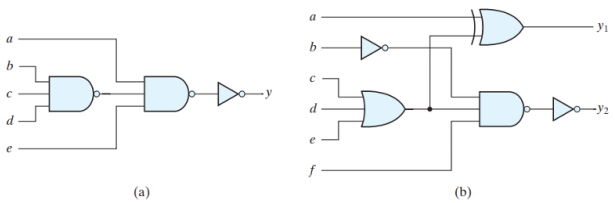


Fig 1: Ejercicio 3

Tabla de Diagrama a:

	A	B	C	D	E	Y
0	0	0	0	0	0	1
1	0	0	0	0	1	1
2	0	0	0	1	0	1
3	0	0	0	1	1	1
4	0	0	1	0	0	1
5	0	0	1	0	1	1
6	0	0	1	1	0	1
7	0	0	1	1	1	1
8	0	1	0	0	0	1
9	0	1	0	0	1	1
10	0	1	0	1	0	1
11	0	1	0	1	1	1
12	0	1	1	0	0	1
13	0	1	1	0	1	1
14	0	1	1	1	0	1
15	0	1	1	1	1	1
16	1	0	0	0	0	1
17	1	0	0	0	1	1
18	1	0	0	1	0	1
19	1	0	0	1	1	1
20	1	0	1	0	0	1
21	1	0	1	0	1	1
22	1	0	1	1	0	1
23	1	0	1	1	1	1
24	1	1	0	0	0	1
25	1	1	0	0	1	1
26	1	1	0	1	0	1
27	1	1	0	1	1	1
28	1	1	1	0	0	1
29	1	1	1	0	1	1
30	1	1	1	1	0	1
31	1	1	1	1	1	0

Funcion diagrama a:

$$y = (((BCD)'AE)')'$$

Tabla de Diagrama b:

A	B	C	D	E	F	Y1	Y2
0	0	0	0	0	0	0	0
0	0	0	0	0	1	0	0
0	0	0	0	1	0	1	0
0	0	0	0	1	1	1	1
0	0	0	1	0	0	1	0
0	0	0	1	0	1	1	1
0	0	0	1	1	0	1	0
0	0	0	1	1	1	1	1
0	0	1	0	0	0	1	0
0	0	1	0	0	1	1	1
0	0	1	0	1	0	1	0
0	0	1	0	1	1	1	1
0	0	1	1	0	0	1	0
0	0	1	1	0	1	1	1
0	0	1	1	1	0	1	0
0	0	1	1	1	1	1	1
0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	1
0	1	0	0	1	0	1	1
0	1	0	0	1	1	1	0
0	1	0	1	0	0	1	0
0	1	0	1	0	1	1	0
0	1	0	1	1	0	1	0
0	1	0	1	1	1	1	0
0	1	1	0	0	0	1	0
0	1	1	0	0	1	1	0
0	1	1	0	1	0	1	0
0	1	1	0	1	1	1	0
0	1	1	1	0	0	1	0
0	1	1	1	0	1	1	0
0	1	1	1	1	0	1	0
0	1	1	1	1	1	1	0
1	0	0	0	0	0	1	0
1	0	0	0	0	1	1	0
1	0	0	0	1	0	0	0
1	0	0	0	1	1	0	1
1	0	0	1	0	0	0	0
1	0	0	1	0	1	0	1
1	0	0	1	1	0	0	0
1	0	0	1	1	1	0	1
1	0	1	0	0	0	0	0
1	0	1	0	0	1	0	1
1	0	1	0	1	0	0	0
1	0	1	0	1	1	0	1
1	0	1	1	0	0	0	0
1	0	1	1	0	1	0	1
1	0	1	1	1	0	0	0
1	0	1	1	1	1	1	0
1	1	0	0	0	0	1	0
1	1	0	0	0	1	1	0
1	1	0	0	1	0	0	0
1	1	0	0	1	1	0	0
1	1	0	1	0	0	0	0
1	1	0	1	0	1	0	0
1	1	0	1	1	0	0	0
1	1	0	1	1	1	0	0
1	1	1	0	0	0	0	0
1	1	1	0	0	1	0	0
1	1	1	0	1	0	0	0
1	1	1	0	1	1	0	0
1	1	1	1	0	0	0	0
1	1	1	1	0	1	0	0
1	1	1	1	1	0	0	0
1	1	1	1	1	1	0	0

Funcion diagrama b:

$$y1 = (C + D + E)B'F$$

$$y2 = A \oplus (C + D + E)$$

Código Verilog Diagrama a:

```
module Ejercicio4_Y(
    input A, B, C, D, E,
    output y
);

assign y = ~(~(~(B & C & D) | (A & E)));

endmodule
```

Código Verilog Diagrama b:

```
module Ejercicio4_Y1Y2(
    input A, B, C, D, E, F,
    output y1, y2
);

assign y1 = ((C | D | E) & (~B & F));
assign y2 = A ^ (C | D | E);

endmodule
```

Ejercicio 6: Del siguiente diagrama de tiempo deduzca la tabla de verdad y la función booleana complementada, siendo A, B y C los bits de entrada y Fs la función de salida. (tome los picos de las entradas y la salida como interferencias indeseadas).

El diagrama temporal se interpretó eliminando los picos indeseados para obtener los valores estables de A, B, C y la salida Fs.
Con esos valores se generó la tabla de verdad y se simplificó la función en SoP mediante Karnaugh. Se implementó posteriormente en HDL para validación.

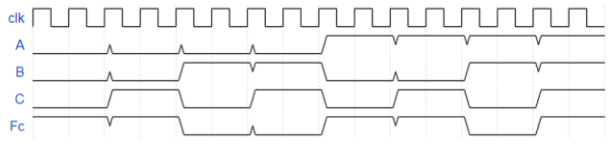


Fig 2: Ejercicio 5

Tabla de Verdad:

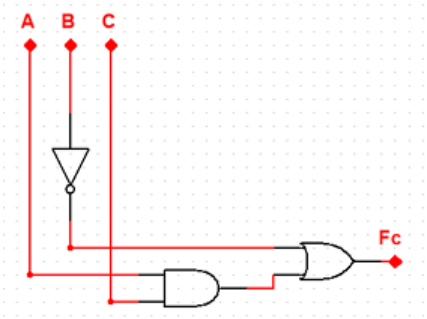
A	B	C	Fc
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

K-Maps y Funciones:

Salida (Fc)				
A/BC	00	01	11	10
0	1	1	0	0
1	1	1	1	0

SoP
B' + AC

Diagrama:



Codigo en Verilog:

```
module ejercicio5(
    input wire A, B, C,
    output wire Fc
);

assign Fc = ~B | A & C ;

endmodule
```

Ejercicio 7: Implementar un contador Jhonson y un anular de tres bits en un software de simulación.

Ambos contadores fueron simulados en software:

- El contador anular generó la secuencia típica de 3 bits con un único "1" desplazándose.
- El contador Johnson generó 6 estados válidos debido a su realimentación invertida.

Diagrama contador Jhonson:

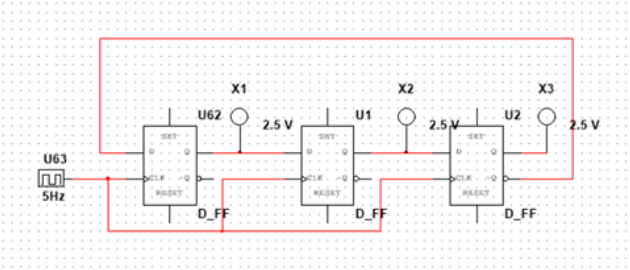
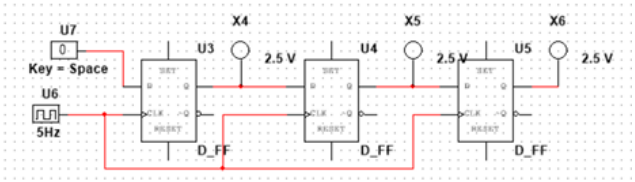


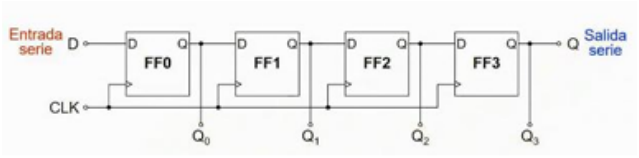
Diagrama contador Anular:



Ejercicio 8: Explique el funcionamiento de un registro Universal, anexe diagrama y defina los diferentes tipos de registros que podemos encontrar.

Se aprendió cual es el funcionamiento del registro universal, incluyendo cada uno de sus modos de operación.

Un registro universal es un tipo de circuito secuencial que se utiliza para guardar información digital y manipularla de diversas formas, dependiendo del modo de operación que se le indique.



Existen varios tipos de registros, cada uno con características específicas:

- **Registro de almacenamiento:** Su función principal es retener datos. La carga se realiza de manera paralela, y no permite desplazamientos.
- **Registro de desplazamiento:** Está diseñado para mover bits en serie, ya sea hacia la izquierda o hacia la derecha. Es útil en tareas como la conversión entre formatos seriales y paralelos o en la implementación de retardos.
- **Registro bidireccional:** Permite el desplazamiento de bits en ambas direcciones, lo que lo hace más flexible que el registro de desplazamiento simple.
- **Registro universal:** Reúne todas las funciones anteriores. Puede cargar datos en paralelo, desplazar bits en ambos sentidos y mantener su contenido sin cambios, según las señales de control que reciba. Por ello, se considera una unidad de almacenamiento temporal altamente adaptable.

Ejercicio 9: Defina que es un ADC (Analog Digital converter) y un DAC anexe explicacion minimo 3 parrafo, anexe fotos y diagramas y 10 aplicaciones donde se utilizan estos conversores. Anexe info sobre un IC comercial de 12 bits que sea un ADC y otro de DAC.

¿Qué es un ADC (Analog-to-Digital Converter)?

Un ADC, o convertidor analógico–digital, es un dispositivo electrónico encargado de transformar una señal analógica —continua en amplitud y tiempo— en una señal digital compuesta por valores discretos. Este proceso es fundamental en sistemas electrónicos modernos, ya que la mayoría de los sensores generan señales analógicas, mientras que los microcontroladores, procesadores y sistemas digitales interpretan únicamente valores binarios. Un ADC permite cuantificar, muestrear y codificar estas señales para que puedan procesarse internamente dentro de dispositivos digitales.
El funcionamiento de un ADC se basa en tres etapas principales: muestreo, donde se toman muestras periódicas de la señal; cuantificación, donde cada muestra se aproxima al nivel digital más cercano; y codificación, donde cada nivel cuantificado se representa mediante un número binario. La resolución del ADC (por ejemplo, 12 bits) determina cuántos niveles de cuantificación existen y qué tan precisa será la conversión de la señal analógica. Un ADC de 12 bits puede representar 4096 niveles distintos, permitiendo una representación más detallada de la señal original. Los ADC se clasifican en distintos tipos según su arquitectura, como los ADC de aproximación sucesiva (SAR), sigma–delta, flash o pipeline. Cada uno presenta ventajas particulares en términos de velocidad, consumo de energía y precisión. En general, los ADC son esenciales en instrumentación, sistemas embebidos y aplicaciones que requieren la digitalización de información del entorno físico.

¿Qué es un DAC (Digital-to-Analog Converter)?

Un DAC, o convertidor digital–analógico, es un dispositivo que realiza el proceso inverso al ADC: toma un valor digital —compuesto por bits— y lo convierte en un nivel analógico proporcional. Los DAC permiten que sistemas digitales produzcan señales analógicas, las cuales pueden representar audio, control de motores, iluminación analógica, señales de radiofrecuencia, entre otros. Sin un DAC, un microcontrolador o FPGA sería incapaz de interactuar con dispositivos que requieren señales continuas.

La operación de un DAC normalmente se basa en una red resistiva ponderada, un convertidor de corriente o arquitecturas sigma–delta. Dependiendo de la resolución (por ejemplo, 12 bits), la salida analógica podrá adoptar un número determinado de niveles posibles; un DAC de 12 bits puede generar 4096 valores analógicos distintos. La linealidad, velocidad de actualización y precisión son parámetros fundamentales que determinan la calidad de conversión.

Los DAC son ampliamente utilizados en sistemas de audio digital, mezcladores de sonido, instrumentación, fuentes analógicas controladas digitalmente, y control de procesos industriales. Su capacidad de transformar señales digitales en valores analógicos precisos los convierte en elementos indispensables dentro de sistemas de control y comunicación.

Aplicaciones comunes donde se utilizan ADC y DAC (10 ejemplos):

1. Sensores de temperatura (ADC) – lectura analógica y procesamiento digital.
2. Sistemas de audio digital (DAC) – conversión de música digital en sonido.
3. Automatización industrial – adquisición de señales de presión, flujo y voltaje.
4. Control de motores (DAC) – generación de señales analógicas de referencia.
5. Instrumentación electrónica – osciloscopios, multímetros digitales.
6. Cámaras digitales (ADC) – conversión de luz captada por el sensor en valores digitales.
7. Sistemas de comunicaciones – modulación y demodulación digital.
8. Impresoras 3D y CNC – control analógico de motores y extrusores.
9. Sistemas de control PID – entradas analógicas sensadas y salidas analógicas de control.
10. Equipos biomédicos – ECG, pulsioxímetros, respiradores, monitoreo de señales fisiológicas.

IC comercial ADC de 12 bits – MCP3201 (Microchip):

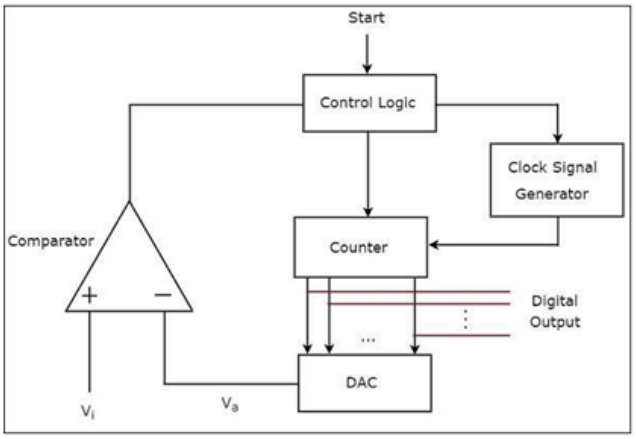
El MCP3201 es un convertidor analógico–digital tipo SAR de 12 bits fabricado por Microchip. Opera mediante interfaz SPI y permite una velocidad de conversión de hasta 100 ksps, siendo ideal para aplicaciones de adquisición de datos, sensores industriales y sistemas embebidos. Funciona con un rango de alimentación de 2.7 V a 5.5 V, lo cual lo hace compatible con la mayoría de microcontroladores modernos. Su arquitectura ofrece excelente relación precisión/consumo energético y permite lecturas estables incluso en entornos con ruido moderado.

IC comercial DAC de 12 bits - MCP4921 (Microchip):

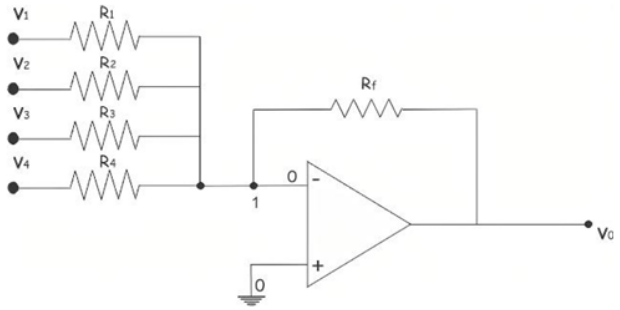
El MCP4921 es un convertidor digital–analógico de 12 bits controlado mediante SPI. Proporciona una salida analógica estable gracias a su arquitectura basada en resistor ladder (R-2R) y ofrece un amplio rango de voltaje de salida configurable mediante una

referencia externa. Este DAC es utilizado en generación de formas de onda, control analógico de motores, sistemas de audio y aplicaciones que requieren variación fina de señal. Su resolución de 4096 niveles permite obtener transiciones suaves y precisas en sistemas de control.

Fotos: ADC



DAC



Ejercicio 10: Realizar un contador que de 4 bits que cuente ascendentemente y luego descienda hasta 4´b0000 en un bucle infinito. Use un flip flop tipo D

En este ejercicio se diseñó un contador secuencial de 4 bits capaz de realizar un conteo ascendente desde 0000 hasta 1111 y luego descender nuevamente hasta 0000 de forma continua. Para ello se elaboró una tabla de estado completa donde se relacionaron todas las combinaciones posibles del contador con su correspondiente estado siguiente, asegurando que la secuencia cambiara automáticamente de ascendente a descendente al alcanzar el valor máximo, y viceversa. Posteriormente, se generaron las ecuaciones de entrada para los flip-flops tipo D mediante la simplificación con Mapas de Karnaugh, obteniendo expresiones lógicas minimizadas que garantizan un diseño más eficiente y estable.

Una vez obtenidas las ecuaciones, el contador fue implementado y verificado en un software de simulación digital, comprobando la correcta alternancia entre la fase ascendente y descendente sin errores de transición. Finalmente, se describió el circuito en lenguaje Verilog HDL, validando su funcionamiento en un entorno de síntesis para FPGA. Este ejercicio permitió reforzar los conceptos fundamentales del diseño secuencial, el uso de flip-flops tipo D y la aplicación de herramientas de simplificación lógica para construir sistemas binarios dinámicos y confiables.

Tabla de estados:

Presente				X	Futuro				D3	D2	D1	D0
A3	A2	A1	A0		A3+	A2+	A1+	A0+				
0	0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	1	1	1	1	1	1	1	1	1
0	0	0	1	0	0	0	1	0	0	0	1	0
0	0	0	1	1	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	1	1	0	0	1	1
0	0	1	0	1	0	0	0	1	0	0	0	1
0	0	1	1	0	0	1	0	0	0	1	0	0
0	0	1	1	1	0	0	1	0	0	0	1	0
0	1	0	0	0	0	1	0	1	0	1	0	1
0	1	0	0	1	0	0	1	1	0	0	1	1
0	1	0	1	0	0	1	1	0	0	1	1	0
0	1	0	1	1	0	1	0	0	0	1	0	0
0	1	1	0	0	0	1	1	1	0	1	1	1
0	1	1	0	1	0	1	0	1	0	1	0	1
0	1	1	1	0	1	0	0	0	1	0	0	0
0	1	1	1	1	0	1	1	0	0	1	1	0
1	0	0	0	0	1	0	0	1	1	0	0	1
1	0	0	0	1	0	1	1	1	0	1	1	1
1	0	0	1	0	1	0	1	0	1	0	1	0
1	0	0	1	1	1	0	0	0	1	0	0	0
1	0	1	0	0	1	0	1	1	1	0	1	1
1	0	1	0	1	1	0	0	1	1	0	0	1
1	0	1	1	0	1	1	0	0	1	1	0	0
1	0	1	1	1	1	0	1	0	1	0	1	0
1	1	0	0	0	1	1	0	1	1	1	0	1
1	1	0	0	1	1	0	1	1	1	0	1	1
1	1	0	1	0	1	1	1	0	1	1	1	0
1	1	0	1	1	1	1	0	0	1	1	0	0
1	1	1	0	0	1	1	1	1	1	1	1	1
1	1	1	0	1	1	1	0	1	1	1	0	1
1	1	1	1	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	0	1	1	1	0

K-Maps y Funciones:

Flip Flop (D3) - A'				
BC/DE	00	01	11	10
00		1		
01				
11				1
10				

Flip Flop (D3) - A				
BC/DE	00	01	11	10
00	1		1	1
01	1	1	1	1
11	1	1	1	
10	1	1	1	1

SoP
AD'E' + AC'D + AB'C + ABE + A'B'C'D'E + A'BCDE'

Flip Flop (D2) - A'				
BC/DE	00	01	11	10
00		1		
01				1
11	1	1	1	
10	1		1	1

Flip Flop (D0) - A'				
BC/DE	00	01	11	10
00	1	1		
01	1	1		
11	1	1		
10	1	1		

Flip Flop (D2) - A				
BC/DE	00	01	11	10
00		1		
01				1
11	1	1	1	
10	1		1	1

Flip Flop (D0) - A				
BC/DE	00	01	11	10
00	1	1		
01	1	1		
11	1	1		
10	1	1		

SoP
BD'E' + BC'D + BCE + B'C'D'E + B'CDE'

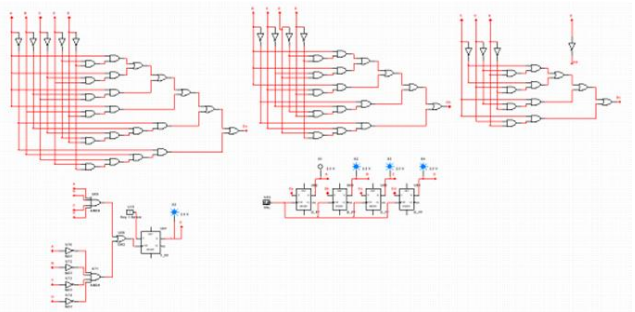
SoP
D'

Flip Flop (D1) - A'				
BC/DE	00	01	11	10
00		1		1
01	1		1	
11	1		1	
10		1		1

Flip Flop (D1) - A				
BC/DE	00	01	11	10
00		1		1
01	1		1	
11	1		1	
10		1		1

SoP
C'D'E + C'DE' + CD'E' + CDE

Diagrama:



Codigo Verilog:

```
module ejercicio9(
    input wire clk,
    output wire A, B, C, D
);
    wire clock ;
        clk U_CLOCK(
        .clock_in(clk),
        .clock_out(clock)
    ) ;
    wire nA, nB, nC, nD, nE
    wire Da, Db, Dc, Dd, E

    //Uso de FF
    ff_d U_da(
        .clk(clock),
        .rst_n(1'b1),
        .D(Da),
        .Q(A)
    );
    not U_NOTA(nA, A) ;

    ff_d U_db(
        .clk(clock),
        .rst_n(1'b1),
        .D(Db),
        .Q(B)
    );
    not U_NOTB(nB, B) ;

    ff_d U_dc(
        .clk(clock),
        .rst_n(1'b1),
        .D(Dc),
        .Q(C)
    );
    not U_NOTC(nC, C) ;

    ff_d U_dd(
        .clk(clock),
        .rst_n(1'b1),
        .D(Dd),
        .Q(D)
    );
    not U_NOTD(nD, D) ;
```

```
    //Funciones
    wire a0, a1, a2, a3, a4, a5 ;
    and U_AND00(a0, A, nD, nE);
    and U_AND01(a1, A, nC, D);
    and U_AND02(a2, A, nB, C);
    and U_AND03(a3, A, B, E) ;
    and U_AND04(a4, nA, nB, nC, nD, E) ;
    and U_AND05(a5, nA, B, C, D, nE) ;
    or U_OR0(Da, a0, a1, a2, a3, a4, a5) ;

    wire b0, b1, b2, b3, b4 ;
    and U_AND06(b0, B, nD, nE);
    and U_AND07(b1, B, nC, D) ;
    and U_AND08(b2, B, C, E) ;
    and U_AND09(b3, nB, nC, nD, E) ;
    and U_AND10(b4, nB, C, D, nE) ;
    or U_OR1(Db, b0, b1, b2, b3, b4) ;

    wire c0, c1, c2, c3 ;
    and U_AND11(c0, nC, nD, E) ;
    and U_AND12(c1, nC, D, nE) ;
    and U_AND13(c2, C, nD, nE) ;
    and U_AND14(c3, C, D, E) ;
    or U_OR2(Dc, c0, c1, c2, c3) ;

    buf U_BUF(Dd, nD) ;


endmodule
```

IV. RESULTADOS Y ANÁLISIS

Los resultados obtenidos a lo largo de esta práctica demostraron el funcionamiento correcto de cada uno de los circuitos combinacionales y secuenciales implementados. En los ejercicios relacionados con multiplexores, tanto el MUX 2:1 como el MUX 4:1 generaron las salidas definidas en las tablas de verdad, validando la capacidad de estos dispositivos para implementar funciones booleanas mediante configuraciones PoS y SoP. El uso de términos don't care permitió optimizar expresiones y reducir la complejidad lógica, algo que se confirmó durante la simulación en el software utilizado.

En los ejercicios secuenciales, la creación de máquinas de estado finito, sistemas de alarma y contadores demostró que las transiciones entre estados se realizaron de manera estable y sin errores. Las FSM implementadas con flip-flops RS siguieron correctamente las secuencias establecidas en sus diagramas, mientras que el contador de 4 bits ascendiente/descendiente implementado con flip-flops tipo D evidenció un comportamiento preciso ante cada pulso de reloj. Asimismo, los contadores Johnson y anulares generaron las secuencias características de sus arquitecturas, validando el uso de realimentaciones y desplazamientos como elementos clave en el diseño digital.

Por otro lado, los ejercicios basados en diagramas lógicos y temporales permitieron verificar que las funciones deducidas coincidían con las salidas esperadas en cada caso, reforzando el proceso de análisis de señales y la interpretación de diagramas funcionales. Finalmente, la implementación de los diseños en HDL mediante Verilog y su síntesis en GoWin IDE confirmaron que el comportamiento simulado coincidía con el comportamiento hardware esperado, demostrando la importancia de combinar teoría, simplificación lógica, simulación y verificación digital para asegurar la correcta operación de los sistemas diseñados.

V. Conclusion

La práctica permitió aplicar de manera integral conceptos esenciales del diseño digital, abordando tanto sistemas combinacionales como secuenciales. A lo largo de los ejercicios se utilizaron diversas herramientas de análisis como tablas de verdad, productos de suma, sumas de productos y Mapas de Karnaugh, así como dispositivos fundamentales como multiplexores y flip-flops.

El uso de software de simulación y la implementación en Verilog HDL facilitaron la comprensión profunda de los circuitos, permitiendo validar en hardware las funciones analizadas teóricamente. Además, se ampliaron conocimientos sobre contadores avanzados, registros universales y conversores analógicos/digitales, temas relevantes en sistemas digitales modernos.

En conjunto, esta práctica fortaleció la capacidad de diseñar, analizar, implementar y documentar sistemas digitales, integrando teoría y aplicación de manera efectiva.

VI. Referencias

https://en.wikipedia.org/wiki/Digital_electronics
https://naylampmechatronics.com/blog/39_resistencias-pull-up-y-pull-down.html
https://es.wikipedia.org/wiki/Sistema_de_numeraci%C3%B3n
https://es.wikipedia.org/wiki/C%C3%B3digo_binario
https://es.slideshare.net/slideshow/sistemas-de-codigos-en-la-electronica-digital/158426000